



RFTM

FMS APIs Guide

Version 8.5.0

History

Date	Version	Comments
2020-05-26	7.6.0	Creation
2021-02-11	7.7.0	V1 - UC OTDR extraction V2 - UC create optical terminal & ports
2021-02-17	7.8.0	Addition of PON use cases Search optical route by name
2021-07-01	7.9.0	Get .sor file binary with base64 option
2021-11-26	7.10.0	RTU logs consulting Notification handler add-on
2022-04-03	7.10.1	Search optical device by name endpoint Configuration/files end point Recovery baseline
2022-06-03	7.11.0	Manage Test Configuration endpoint Configuration end point
2023-01-18	7.13.0	Get users through Service Authorization Proxy
2023-05-24	7.14.0	Adding fields deprecation notice for RTU registration API
2023-07-18	7.14.1	Add modules sync and diff endpoint.
2023-11-01	7.14.2	1. Updated TOD endpoint for FG-750. 2. Get PDF report of OTDR measurement.
2023-12-20	7.14.3	Tag/untag measurement of test type VeryFirstReference or Adhoc as birth certificate.
2024-03-15	7.14.3.2	Highlight the contractual requirements on the proper use of http headers
2024-05-08	8.0.0	Add client_secret and scope attribute in the prerequisites. Remote iOLM test changes done considering zero position on the remote switch. OwningDiagram field mandatory in RTU creation
2024-06-17	8.1.0	Add new diagram content endpoints
2025-02-07	8.4.0	Add optical routes notes management API
2025-12-12	8.5.0	Integration of ims-alpha addon

Table of content

- [History](#)
- [Purpose](#)
 - [API access prerequisites](#)
 - [Reference documents](#)
 - [Postman environment variables](#)
- [API call header](#)
- [Topology API](#)
 - [Roles](#)
 - [USE CASE - How to register RTUs at the FMS central site](#)
 - [USE CASE - How to add or remove ROTAs on an attached RTU \(applies to EXFO FG-750\)](#)
 - [USE CASE - How to create an optical route to an RTU](#)
 - [USE CASE - How to manage notes for an optical route](#)
 - [USE CASE - How to get OpticalRoutes from partial OpticalRouteName \(new in 7.8\)](#)
 - [USE CASE - How to get optical devices from partial name \(new in 7.10.1\)](#)
 - [USE CASE - How to create an optical terminal and link an optical device port to an optical route](#)
 - [USE CASE - Adding/Importing and Managing "TestConfigurations", a template for testsetups](#)
 - [USE CASE - How to set up and run an OTDR test on point to point optical route \(applies to EXFO FG-750\)](#)
 - [USE CASE - How to set up and run an IOLM test on point to point optical route \(p2p as type of route in FMS\)](#)
 - [Overview - How to run test on PON optical route](#)
 - [USE CASE - How to run Ad Hoc iOLM or OTDR test](#)
 - [Overview - When to update test setup of a route](#)
 - [USE CASE - Change link definition in test setup and update baseline](#)
 - [USE CASE - Change thresholds in test setup](#)
 - [USE CASE - Change "Golden Trace" pulse and duration and update baseline](#)
 - [USE CASE - How to launch a HRD test using OpticalRouteName](#)
 - [USE CASE - Moving an HRD to another terminal port](#)
 - [USE CASE - Basic recovery baselines and HRDs after link repair](#)
 - [USE CASE - Diagram populating and content retrieving](#)
 - [USE CASE - Domain configuration for alarm notification](#)
 - [USE CASE - To calculate and send LinkLength and LinkLoss value for RemoteiOLM test based on zero position harmonization](#)
- [Measurement data & measurement files APIs](#)
 - [Roles](#)
 - [USE CASE - How to view the measures - Result Consulting](#)
 - [USE CASE - How to upload a zip with the measures related to your current measure](#)
 - [USE CASE - How to retrieve the .sor file in binary or Base64 from a resultId of an OTDR result](#)
 - [USE CASE - How to retrieve the .pdf file from a resultId of an OTDR result](#)
 - [USE CASE - To Add / update birth certificate of an OTDR result](#)
 - [Roles](#)
- [Alarm API](#)
 - [Roles](#)
 - [Alarms consulting](#)
- [RTU logs consulting](#)
 - [Upload logs from RTU to file system](#)
 - [Change the log level verbosity](#)
- [Notification handler addon API](#)
 - [Get all notifications](#)
 - [Get notification related to a promiseId](#)

- [Postman collection](#)
- [Service Authorization Proxy](#)
 - [IAM](#)
- [Topology V2 API](#)
 - [Inventory Management](#)

Purpose

This guide refers to the APIs exposed by FMS. The APIs are documented with swagger and made available.

Several Use Cases or workflows are described to exploit these APIs. Postman collections will be attached to this document to test the described use cases.

You can also use Daniel Stenberg's cURL command line software. It is available at the following address: <https://curl.haxx.se/> (<https://curl.haxx.se/>).

API access prerequisites

Access to the API is secure and requires certain user roles.

Creation and modification endpoints are granted to any user account with the role of master.

Get endpoints are granted to any user account with the role of read except for the user endpoint which contains sensitive data on which master rights will be required as well.

The access token expires after 15 minutes by default.

The screenshot shows the EXFO Fiber configuration interface. On the left is a dark sidebar with a menu. The main area is titled 'Fiber' and has several tabs: General, Login, Keys, Email, Themes, Localization, Cache, and Tokens. The 'Tokens' tab is selected. It contains various settings for session and token management, each with a help icon (question mark) and a default value.

Setting	Value	Unit
Default Signature Algorithm	RS256	
Revoke Refresh Token	OFF	
SSO Session Idle	7	Days
SSO Session Max	7	Days
SSO Session Idle Remember Me	0	Minutes
SSO Session Max Remember Me	0	Minutes
Offline Session Idle	31535999	Minutes
Offline Session Max Limited	OFF	
Client Session Idle	0	Minutes
Client Session Max	0	Minutes
Access Token Lifespan	15	Minutes
Access Token Lifespan For Implicit Flow	5	Days

You can update the access token lifespan according to your needs (open the keycloak admin console).

To request an access token, use the following endpoint:

```
curl -X POST https://__<<auth.fms.local>>__/auth/realms/Fiber/protocol/openid-connect/token -H 'Content-Type: application/x-www-form-urlencoded' \
-d 'client_id=fg-topologyapi&grant_type=password&username=__<<your_username>>__&password=__<<your_password>>__&client_secret=__<<client_secret>>__'
```

Change parameters to adjust to environment:

Parameter	Description
<<auth.fms.local>>	Address of the authentication (IAM) sub-system
<<your_username>>	User name used to access FMS application that has the fg-topology-master role associated. Can be an email address.
<<your_password>>	Password corresponding to above user name used to access FMS application.

Parameter	Description
<<client_secret>>	client_secret corresponding to IAM_CLIENT_SECRET secret.

This request will return an access|token as per the following example:

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICJ...55FwQi7gEw",
  "expires_in": 900,
  "refresh_expires_in": 604800,
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICI2MD...1yi5aY",
  "token_type": "Bearer",
  "not-before-policy": 0,
  "session_state": "68d8e8e2-a25b-4cc2-a1c2-d913866ccd6f",
  "scope": "profile email fms_client_scope"
}
```

This access_token value can be used in sub-sequent calls. By default, this token is valid for 15 minutes. Once expired, the API will reply with 401 response code.

You have the same example with the postman collections associated to this document.

Reference documents

Some files and documents are available to provide examples and description of parameters

- Postman collections (Examples of API calls with specific parameters and payloads)
- FMS PON measurement payloads and results.docx ()

Postman environment variables

Before loading the environment file to postman, update the environment variables.

Replace xxxx with the good domain.

File name: postman_environment_template.json

API call header

Make sure to use the correct value : **'application/json'** in the required http requests headers ("Content-Type", "Accept") and not another value such as **'text/plain'** which could work but eventually causes issues in some test cases.

Topology API

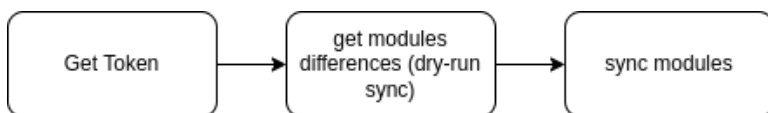
Roles

Role Name	Composite	Description
fg-topology-domain-access	True	Domain specific read access
fg-topology-master	True	Full access
fg-topology-read	True	Read only access

Note: *domain* is an authorization down to infrastructure/object level which requires user(s) to be associated to an existing domain for this role to apply.

USE CASE - How to register RTUs at the FMS central site

Workflow



Steps

1 - Get modules differences

Steps

1 - RTUs registration

Endpoint: [#/control/getRegisteredRtus \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/getRegisteredRtus\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/getRegisteredRtus)

Through DNS resolution using SRV records, the RTU will register itself to FMS.

It is possible to list all RTUs that have registered.

It allows to validate that the previous configuration done on the RTU and the system configuration (DNS, NTP) are correct.

Thanks to this automatic registration, FMS knows the whole system configuration of an RTU (software version, IP addresses...).

The registration Key and serial number fields are important and allow to declare and attach the logical RTU in FMS.

Deprecated fields:

The fields `model`, `serialNumber`, `hostName`, `testCoordinatorVersion`, `apiVersion`, `hardwareInfo`, `capabilities`, `versions` in the response are deprecated. Same information is available serialized in the field `hardwareInfoObject`.

A query boolean parameter `showSerialized`, defaulting to `true`, enables to keep these deprecated fields in the response. If `false`, they are only available within the field `hardwareInfoObject`.

2 - Creating a logical RTU in FMS

Endpoint: [#/remotetestunits/createRtu \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/remotetestunits/createRtu\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/remotetestunits/createRtu)

Creating this device in FMS will make it possible to create the other topology elements that are linked to this RTU (cable segments, optical routes, etc.).

The logical RTU must be associated with a diagram; field `owningdiagram` is mandatory.

See [USE CASE - Diagram populating and content retrieving](#)

Use the serial number field to link the logical RTU with the physical RTU.

Fields:

- **name** and **description** are given by the user
- **model**: RTU-2, OTH-7000, FG-750ST, FG-750EX
- **serialNumber**: retrieved in step 1
- **owningdiagram**: id of diagram owning the RTU

Response:

Get the optical device ID in response to target your RTU in the next steps or another use case.

3 - Attaching RTU- a logical RTU in FMS

Endpoint: [#/control/attachRtu \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/attachRtu\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/attachRtu)

The attach process allows to synchronize the RTU with FMS. This is when the FMS is able to drive the RTU.

FG750 Case: The RTU FG can work on standalone. After the attachment process, the RTU is set to read-only mode and leaves the commands to the central server for remote control.

All local RTU configuration is sent to the central server.

Needs:

- **rtuld** retrieved in the previous step
- **registrationkey** retrieved in the first step

Response:

A summary of applied changes that impact configuration is returned: OTDR and Switches modules:

- **DiscoveredModules:** hardware modules that were not defined before attaching and were added to the configuration
- **ReplacedModules:** pre-configured modules with expected type and serial numbers updated by the physical module characteristics
- **UnmatchedModules:** hardware modules that were initially configured but not matched with modules physically located on RTU

4 - Getting RTU modules and ports

Endpoints:

- [#/remotetestunits/getModules](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/remotetestunits/getModules) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/remotetestunits/getModules)
- [#/remotetestunits/getPorts3](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/remotetestunits/getPorts3) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/remotetestunits/getPorts3)

Allows you to retrieve the configuration of the modules/ports of your RTU.

Response:

An array with all modules/ports' details (type, ID, ports...).

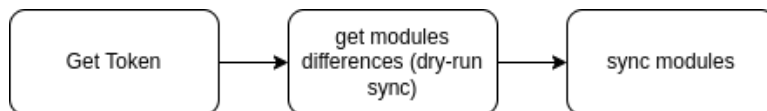
The port ID is important to link RTU to an optical route.

Postman collection

File name: UC_Topo_RTU_Configuration.postman_collection.json

USE CASE - How to add or remove ROTAUs on an attached RTU (applies to EXFO FG-750)

Workflow



Steps

1 - Get modules differences (dry-run sync)

Endpoint: [#/control/dryRunsyncModules](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/dryRunsyncModules) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/dryRunsyncModules)

Dry run synchronization of modules of an attached Topology RTU. Read modules data from the physical RTU and report differences with the logical one in FMS.

⚠ Limitation: Currently, this endpoint is only supported for FG-750s.

Response:

A summary of differences that impact configuration is returned, similar to the result of a RTU attach.

- **DiscoveredModules:** hardware modules that were not defined before attaching and were added to the configuration
- **ReplacedModules:** pre-configured modules with expected type and serial numbers updated by the physical module characteristics
- **UnmatchedModules:** hardware modules that were initially configured but not matched with modules physically located on RTU

2 - Synchronize modules

Endpoint: [#/control/syncModules](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/syncModules) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/syncModules)

Synchronize modules of an attached Topology RTU. Read modules data from the physical RTU and apply changes to the logical one in FMS.

△ Limitation: Currently, the supported operation are restricted to add or remove ROTAU modules on a FG-750.

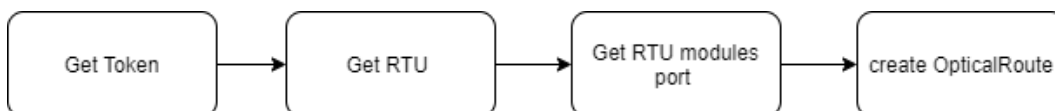
Response:

A summary of applied changes that would impact configuration is returned, similar to the result of a RTU attach. Only ROTAU are present in this response.

- **DiscoveredModules:** hardware modules that were not defined before attaching and were added to the configuration
- **ReplacedModules:** pre-configured modules with expected type and serial numbers updated by the physical module characteristics
- **UnmatchedModules:** hardware modules that were initially configured but not matched with modules physically located on RTU

USE CASE - How to create an optical route to an RTU

Workflow



Steps

1 - Getting RTU

Endpoint: [#/remotetestunits/getRtus](#) (https://d92waqemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/remotetestunits/getRtus)

Retrieves remote test units (see swagger for detailed information on this end point).

Response:

Contains the information related on RTU (ID, serialnumber, module information with their ID...).

2 - Getting RTU modules port

Endpoints: [#/remotetestunits/getModules](#) (https://d92waqemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/remotetestunits/getModules)

```
[ \#/remotetestunits/getPorts3] (https://d92waqemn7fj.cloudfront.net/topology-api/master/latest/index.html#/remotetestunits/getPorts3)
```

Allows you to retrieve the configuration of the modules/ports of your RTU.

Response:

An array with all modules/ports' details (type, ID, ports...).

The port ID is important to link RTU to an optical route.

3 - Cascading module connection

Endpoint: [#/remotetestunits/getRtuPortsAvailability](#) (https://d92waqemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/remotetestunits/getRtuPortsAvailability)

It may also be interesting to look at the port sequencing for a cascading configuration before creating an optical route because the sequencing must be respected.

4 - Creating an Optical Route

In order to create an Optical Route, you need the ports ID and the RTU ID.

Endpoint: [#/opticalroutes/createOpticalRoute](#) (https://d92waqemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/createOpticalRoute)

Fields:

- **name** and **description** are given by the user,
- **type** can be:
 - *PonOpticalRoute* for PON routes

- *OpticalRoute* for P2P routes
- **linkType** can be:
 - *DARK* for dedicated surveillance fiber
 - *LIVE_TAM* for WDM accessed live fiber
- **rtuld**
- **rtuPorts** is a sorted sequence of ports starting with OTDR module port ID followed by port IDs on each cascaded switch.

Response:

The sequencing must be respected when creating the optical route otherwise you will have an error like this:

```
"error": {
  "code": 400,
  "message": "Invalid monitored asset port sequence. Port 793 must be preceded by port 790 and followed by no devices.",
  "messageKey": "41042_invalid_previous_port_and_following_device",
  "parameters": [
```

This call will return the created optical route with its unique ID.

The returned optical route ID is important to configure the tests on this optical route.

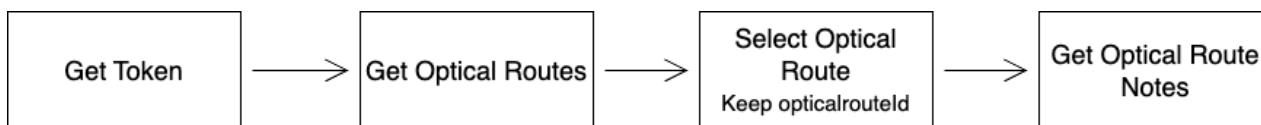
If the RTU is attached, the default test setup is automatically created.

Postman collection

File name: UC_Topo_OpticalRoute_Configuration.postman_collection.json

USE CASE - How to manage notes for an optical route

Workflow



Steps

1 - Getting optical route ID

If the ID of targeted optical route is unknown, you can retrieve a list of optical routes using:

Endpoint: GET /api/topology/opticalroutes

Headers required:

- Authorization: Bearer {access_token}
- Content-Type: application/json
- Accept: application/json

Response format:

```
{
  "objects": [
    {
      "opticalRoute": {
        "id": 1092,
        "name": "WRoute",
        "type": "OpticalRoute"
        // ... other properties
      }
    }
  ],
  "totalPageCount": 1,
  "totalObjectCount": 1
}
```

From the response, identify the opticalRouteId from the opticalRoute.id property.

2 - Retrieving notes

Endpoint: GET /api/topology/opticalroutes/{opticalRouteId}/notes

Retrieves all notes associated with a specific optical route.

Headers required:

- Authorization: Bearer {access_token}
- Content-Type: application/json
- Accept: application/json

Query Parameters:

- last (optional): Number of most recent notes to return
 - Must be a positive integer
 - If omitted, returns up to 100 notes

Response Codes:

- 200: Success
- 400: Invalid 'last' parameter
- 404: Optical route not found

Response format: Returns an array of notes.

3 - Creating a note

Endpoint: POST /api/topology/opticalroutes/{opticalRouteId}/notes

Creates a new note for the specified optical route.

Headers required:

- Authorization: Bearer {access_token}
- Content-Type: application/json
- Accept: application/json

Request Body:

```
{  
  "note": "This is the Api Note for Optical Route"  
}
```

Response Codes:

- 201: Successfully created
- 404: Optical route not found
- 500: Internal server error

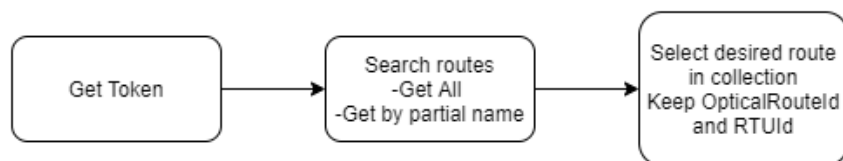
Postman collection

File name: UC_Topo_OpticalRoute_Note.postman_collection.json

The collection includes automatic token management and validation tests for all operations.

USE CASE - How to get OpticalRoutes from partial OpticalRouteName (new in 7.8)

Workflow



Steps

1- Getting Optical Route

Search your Optical Route with a get all command.

Endpoint: [#/opticalroutes/getOpticalRoutes2](#) (https://d92waqemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoutes2)

By default, without parameter, the request applies a get all command but it is possible to apply a filter to reduce the volume of data or search a specific data.

Fields:

- **name:** This string parameter allows to limit the number of returned results. The pattern of filtering used here is the contains mode. A full or a partial name can be used.
- **pageSize:** number max of Optical route returned by page
- **pageNumber:** page number

Example with this following OR list:

OR n°1= 002-WBX4.6-SACRAMENTODT_5G062

OR n°2=002-WBX4.6-SACRAMENTODT_5G063

OR n°3=007-WBX4.7-DEMOODT_5G032

Example 1: Search by full name "002-WBX4.6-SACRAMENTODT_5G063" return only the OR no. 2.

Example 2: Search by partial name start with "002" return the ORs no.1 and 2.

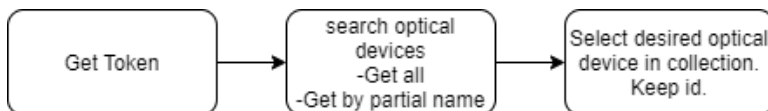
Example 3: Search by partial name containing "ODT" return all OR.

Postman collection

File name: UC_Topo_ORsearch_ByName.postman_collection.json

USE CASE - How to get optical devices from partial name (new in 7.10.1)

Workflow



Steps

1- Getting Optical Device

Search your Optical Device with a get all command.

Endpoint: [#/opticaldevices/getOpticalDevices1](#) (https://d92waqemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticaldevices/getOpticalDevices1)

Retrieves all optical devices up to max_get_size elements. Without pagination, it returns an array of Optical devices:

```
[
  {
    "opticalDevice": {...}
  }
]
```

For get request without parameter ('getAll'), two headers are added in the response:

- 'TotalObjectsCount': indicates the total number of optical devices in the database.
- 'PartialResult': indicates whether or not all elements have been returned. Since getAll request is limited by max_get_size, the field 'PartialResult' will be set to true if the response has been capped.

Using pagination, it returns a json array of 'objects' containing:

- a list of optical devices.
- totalObjectCount: indicates the total number of optical devices present in the database.
- totalPageCount: indicates the total number of pages, presuming each page has max_get_size elements.

```
{
  "objects": [
    {
      "opticalDevice": {...}
    }
  ],
  "totalPageCount": int,
  "totalObjectCount": int
}
```

If pageSize is higher than max_page_size, it returns HTTP 400. max_get_size and max_page_size can be configured/overridden with config file /opt/fms/solution/config/topology/endpoints_size.properties.

Fields:

- **importKey:** import key.
- **ids:** comma-separated list of ids.
- **name:** This string parameter allows to limit the number of returned results. For performance reason, contrary to optical route search, the pattern of filtering used here is the 'begins with' mode. The filtering is not case-sensitive. A full or a partial name can be used.
- **pageSize:** maximum number of devices returned by page.
- **pageNumber:** page number. First page has number 1.

Example with this following optical device list:

device 1 = 002-WBX4.6-SACRAMENTODT_5G062

device 2 = 002-WBX4.6-SACRAMENTODT_5G063

device 3 = 007-WBX4.7-DEMOODT_5G032

Example 1: The search by full name "002-WBX4.6-SACRAMENTODT_5G063" returns only the device no. 2.

Example 2: The search by partial name starting with "002" returns the devices no.1 and 2.

Example 3: The search by partial name containing "ODT" returns no device.

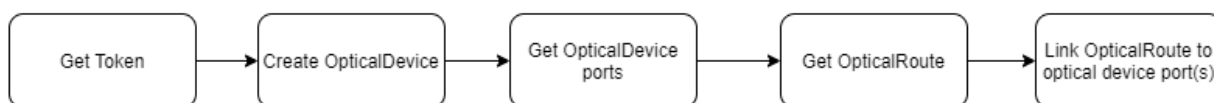
Example 4: The search by partial name containing "00" returns all devices.

Postman collection

File name: UC_Topo_OpticalDevice_Search_ByName.postman_collection.json

USE CASE - How to create an optical terminal and link an optical device port to an optical route

Workflow



Steps

1 - Creating an Optical terminal

Endpoint: [#/opticaldevices/createOpticalDevice \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticaldevices/createOpticalDevice\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticaldevices/createOpticalDevice)

Fields:

- **name** and **description** are given by the user
- **portCount:** number of device ports
- **type:** optical device type (OpticalTerminal, PatchPanel, Splitter, SpliceBox, WavelengthDivisionMultiplexer)
- **siteId (optional):** it is possible to link a device to a site. Prerequisite: Search or create a site before and get the siteId.

Response:

This call will return the created optical device with its unique ID.

2 - Getting optical device ports

Endpoints: [#/opticaldevices/getPorts1](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticaldevices/getPorts1)

Allows you to retrieve the configuration of the ports of your optical device.

Response:

An array with all port's details (ID, deviceId, number, crossingOpticalRoutes...).

The port ID is important to link a device port to an optical route.

3 - Getting optical route

If the ID of targeted optical route is unknown, search your Optical Route with this endpoint, otherwise you can skip this step:

Endpoint: [#/opticalroutes/getOpticalRoutes2](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoutes2)

4 - Linking an optical device port to an optical route

a/ From put optical route endpoint, if you want link several device ports with a single request

Endpoint: [#/opticalroutes/putOpticalRoute](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/putOpticalRoute)

Set the field crossedOpticalDevicePorts of optical route

Example:

"crossedOpticalDevicePorts": [

{

"number": 1, => port number

"deviceId": 698992, => ID of the optical device

"id": 698994 => ID of the optical device port

},

{

"number": 2,

"deviceId": 698992,

"id": 698995

}

],

b/ From put optical device endpoint, if you want link a single device port to an optical route

Endpoint: [#/opticaldevices/putPort1](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticaldevices/putPort1)

Set the field crossingOpticalRoutes of optical device

Example:

"crossingOpticalRoutes": [

595200 => optical route ID

]

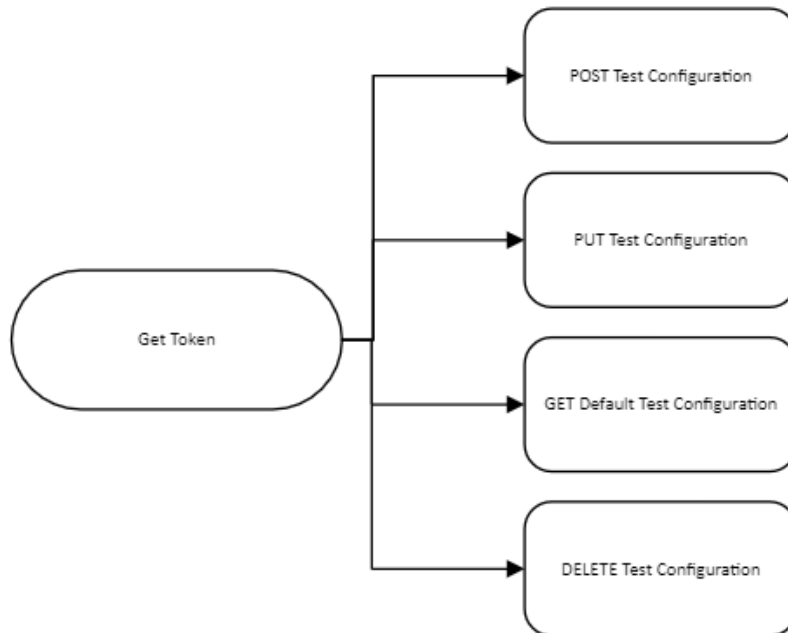
Postman collection

File name: UC_Topo_OpticalDevice_Configuration.postman_collection.json

USE CASE - Adding/Importing and Managing "TestConfigurations", a template for testsetups

This endpoint is to manage configuration templates.

Workflow



Steps

1- Creating TestConfig

Endpoint: [#/testconfigurations/postTestConfiguration](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/testconfigurations)

Note: This end point can be used to import existing customized test configurations files. In that case, the user can copy the file's content in the "payLoad" field.

Response:

Contains the newly created test configuration.

2 - Changing the test config with new settings

Endpoint: [#/testconfigurations/putTestConfiguration](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/testconfigurations)

If a user wants to make test config as default test config, he or she can use this endpoint with field "isDefault": true.

Response:

Contains the updated test configuration.

3 - Fetching default test config for monitoringType (p2p or pon) and testCategory (monitoring or adhoc)

Endpoint: [#/testconfigurations/default?monitoringType=p2p&testCategory=monitoring](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/testconfigurations)

Parameters:

- Key: monitoringType, Value: p2p or pon
- Key: testCategory, Value: monitoring or adhoc

Response:

Contains the default test configuration.

4 - Delete test configuration

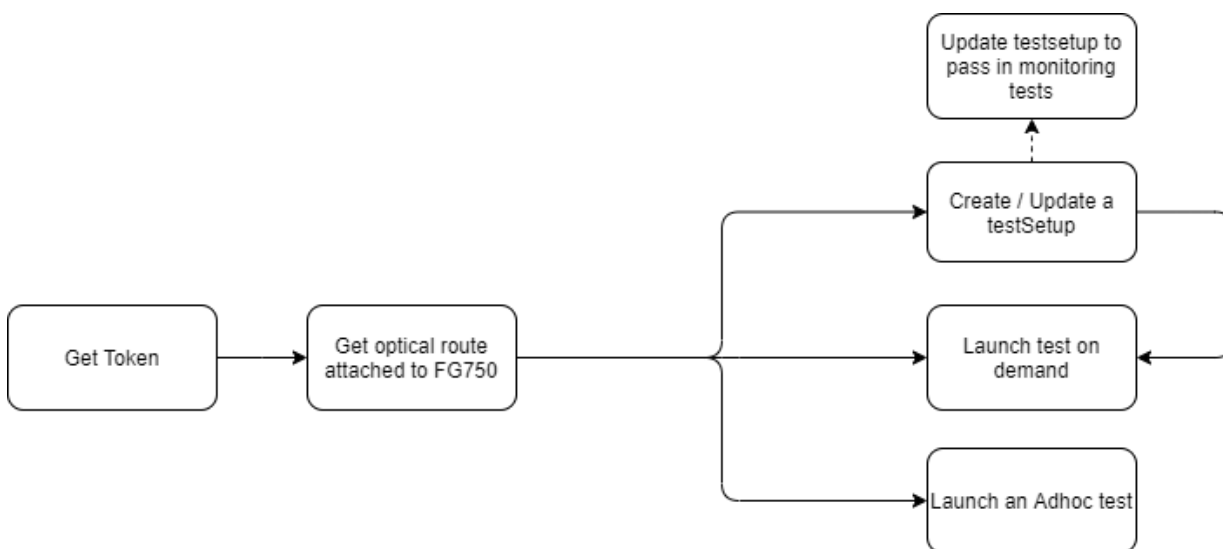
Endpoint: [#/testconfigurations/default?monitoringType=p2p&testCategory=monitoring](#)
(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/testconfigurations)

Default and SystemDefault test configs cannot be deleted.

Postman collection

File name: UC_Topo_TestConfiguration.postman_collection.json

USE CASE - How to set up and run an OTDR test on point to point optical route (applies to EXFO FG-750)

Workflow**Steps****1- Getting optical route**

If the ID of targeted optical route is known, use the following endpoint to have details on your route:

Endpoint: [#/opticalroutes/getOpticalRoute](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoute)

Else, search your Optical Route with this endpoint:

Endpoint: [#/opticalroutes/getOpticalRoutes2](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoutes2)

2.a - Launching an Adhoc test

Endpoint: [#/control/commandOtdrAdHocTest](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandOtdrAdHocTest)

This is a test to be started manually to validate the link with a different setting than the usual monitoring values (with a different pulse value for example).

Check the values configured on FMS. The values can be obtained with the following endpoint, as field "otdrTestSetupDefaultValues" of the returned object:

[#/configuration/getConfigurationFilesContent \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/configuration/getConfigurationFilesContent\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/configuration/getConfigurationFilesContent)

or at [#/configuration/getOtdrTestSetupDefaultValues \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/configuration/getOtdrTestSetupDefaultValues\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/configuration/getOtdrTestSetupDefaultValues)

2.b - Getting test setups

Allows you to retrieve the configuration of tests.

Endpoint: [#/opticalroutes/getTestSetups \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getTestSetups\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getTestSetups)

Creating a test setup

Endpoint: [#/opticalroutes/createTestSetup \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/createTestSetup\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/createTestSetup)

An optical route accepts many test setups.

The default values configured on FMS can be obtained with the following endpoint:

[#/configuration/getConfigurationFilesContent \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/configuration/getConfigurationFilesContent\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/configuration/getConfigurationFilesContent)

For OTDR test, it is possible to configure a test very finely:

You can configure:

- The Type of Test (value domain: Monitoring (by default), ProactiveMaintenance, TestOnDemand)

Property	Description	Type	Mandatory
Type	Test setup type	Text: value domain: - Monitoring - ProactiveMaintenance - TestOnDemand	Yes (default monitoring)
Wavelength	One of the available wavelengths from the OTDR module on the RTU	Wavelength value	Yes
Resolution	Either normal or high	Text: value domain - Normal - High	Yes (default normal)
Range	Test range in meters	Number: 1000 to 320000	No
Pulse	Pulse in seconds as defined by the OTDR module hardware	Number 0.000000001 to 1 (9 decimals)	No
Duration	Test duration in seconds	Number: 3 to 180	No
Auto settings	Lets the system automatically determine the range, pulse and duration	Boolean	Yes (default true)
Data Upload Policy	Specifies to upload or not the binaries files to the server	Boolean	Yes (default true)
Targeted Learning Count	Number of learning traces to take before monitoring	Number: 1 to 9999	Yes (default 30)
Learning be reset	Indicates if the learning will be reset in the future: by a specific date or after a specific period of time.	Boolean	Yes (default false)
Reset At	Date and time on which to refresh the learning traces	Date and time	No
Reset Frequency	Number part of the periodicity of learning refresh	Number: 1 to 999	No
Reset Periodicity	Unit part of the periodicity of learning refresh	Text value domain: - Days - Weeks - Months - Years	No
SpliceLossThreshold	threshold in dB	Number: 0.01 to 5	Yes (default 0.02)
ReflectanceThreshold	threshold in dB	Number: -78 to -14	Yes (default -72)
EndoffiberThreshold	threshold in dB	Number: 1 to 25	Yes (default 4)
ExtendedMode	Enables extended mode	Boolean	Yes (default false)
FaultSectionLoss	Fault detection threshold in dB	Decimal: from 0.01 to 10	Yes, in absence of linked threshold set
FaultEventReflectanceDegradation	Fault detection threshold in dB	Decimal: from 0.1 to 100	Yes, in absence of linked threshold set
FaultEventLoss	Fault detection threshold in dB	Decimal: from 0.01 to 10	Yes, in absence of linked threshold set
FaultInjectionLevel	Fault detection threshold in dB	Decimal: from 1 to 5	Yes, in absence of linked threshold set
FaultLinkTotalLoss	Fault detection threshold in dB	Decimal: from 0.1 to 10	Yes, in absence of linked threshold set

- The ThresholdSet

Four thresholdsets are defined in FMS. In addition to the four (4) provided by default OTDR threshold sets, the system allows to add threshold sets (refer to administration guide).

[#/configuration/getThresholdDefaultValues \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/configuration/getThresholdDefaultValues\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/configuration/getThresholdDefaultValues)

- The Learning expansion

Property	Description	Domain Value	Mandatory
Learning be expanded	Defines if the learning will be expanded or not	Boolean	Yes (default: true)
Expand at	Expands learning at a specific date	Date and time	No
Expand frequency	Number portion of the expansion periodicity	Number: 1 to 999	No
Expand periodicity	Time unit portion of the expansion periodicity	Text value domain: • Days • Weeks • Months • Years	No
Expand step	Number of learning acquisition to perform at each learning expansion	Number: 1 to 9999	Yes (default: 1)
Expand limit	Number of learning traces to be kept	Number: 1 to 9999	No

The default value for the field "duration" is null, but this is configurable (see administration guide). Versions 3.11 and prior of FG-750 allow only null as duration value for a test setup creation if autoSettings is true. On version 3.12 of FG-750, the duration is allowed to be null or to have any value, in all cases. On version 3.13 of FG-750, the duration field is mandatory in all cases.

Updating a test setup

Endpoint: [#/opticalroutes/putTestSetup](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/putTestSetup)

If you want to change some criteria on your test setup, it is possible with this endpoint.

2.c / 3 Launching test on demand

Endpoint: [#/control/OnDemandTest](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/onDemandTest)

This type of test is conducted to check the health of fibers when required.

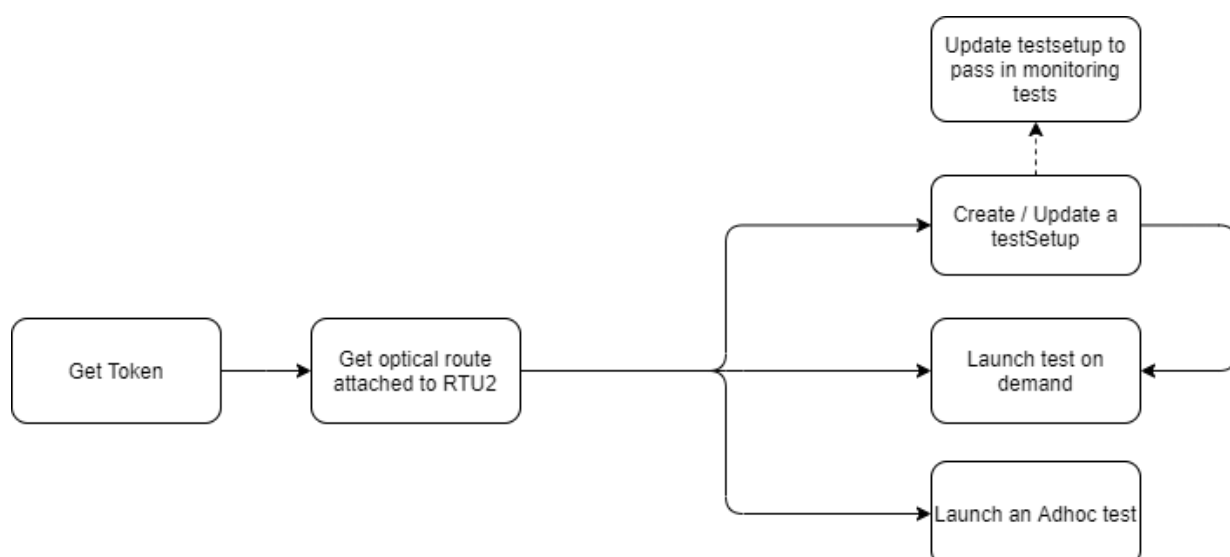
A test on demand shall be launched on a specific test setup. You can retrieve the test setup ID at the step 2b if it is unknown.

Postman collection

File name: UC_Topo_OTDR_Tests_P2P.postman_collection.json

USE CASE - How to set up and run an IOLM test on point to point optical route (p2p as type of route in FMS)

Workflow



Steps

1- Getting optical route

If the ID of targeted optical route is known, use this endpoint to have details on your route:

Endpoint: [#/opticalroutes/getOpticalRoute](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoute) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoute)

Else, search your Optical Route with a get all command.

Endpoint: [#/opticalroutes/getOpticalRoutes2](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoutes2) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoutes2)

2.a - Launching an Adhoc test

Endpoint: [#/control/commandIolmAdHocTest](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandIolmAdHocTest) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandIolmAdHocTest)

This type of test is conducted to check the health of fibers when required.

2.b - Getting test setups

Allows you to retrieve the configuration of tests.

Endpoint: [#/opticalroutes/getTestSetups](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getTestSetups) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getTestSetups)

Creating a test setup

Endpoint: [#/opticalroutes/createTestSetup](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/createTestSetup) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/createTestSetup)

An optical route accepts many test setups.

In Iolm, it is not easy to configure a test Setup.

All parameters are in payload. Parameters are not interpreted and not verified by the server.

The configuration is pushed on the RTU-2.

Updating a test setup

Endpoint: [#/opticalroutes/putTestSetup](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/putTestSetup) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/putTestSetup)

If you want some criteria on your test setup, it is possible with this endpoint.

To pass in monitoring test, just change the parameter isMonitoring to True.

The default payload can be obtained at this endpoint: [#/testconfigurations/default?monitoringType=pon&testCategory=monitoring](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/testconfigurations/default?monitoringType=pon&testCategory=monitoring) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/testconfigurations/default?monitoringType=pon&testCategory=monitoring)

or pass p2p to field monitoringType at this endpoint: [#/testconfigurations/default?monitoringType=p2p&testCategory=monitoring](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/testconfigurations/default?monitoringType=p2p&testCategory=monitoring) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/testconfigurations/default?monitoringType=p2p&testCategory=monitoring)

2.c / 3 - Launching test on demand

Endpoint: [#/control/commandIolmOnDemandTest](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandIolmOnDemandTest) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandIolmOnDemandTest)

A test on demand shall be launched on a specific test setup. You can retrieve the test setup ID at the step 2b if it is unknown.

Postman collection

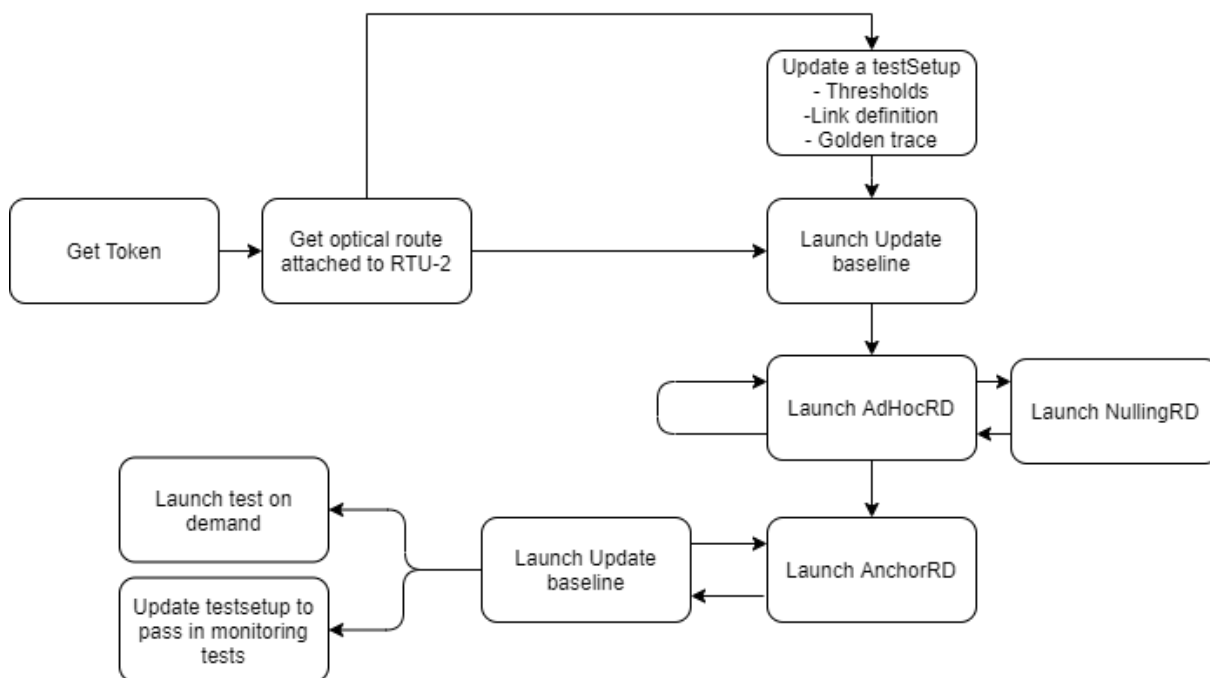
File name: UC_Topo_iOLM_Tests_P2P.postman_collection.json

Overview - How to run test on PON optical route

PON/P2MP optical routes provides additional functionalities with High Reflectance Demarcation (HRD) testing. Here is an overview of the life cycle.

Different use cases are explained below: HRD test for build network & HRD monitor for operate network.

General Workflow



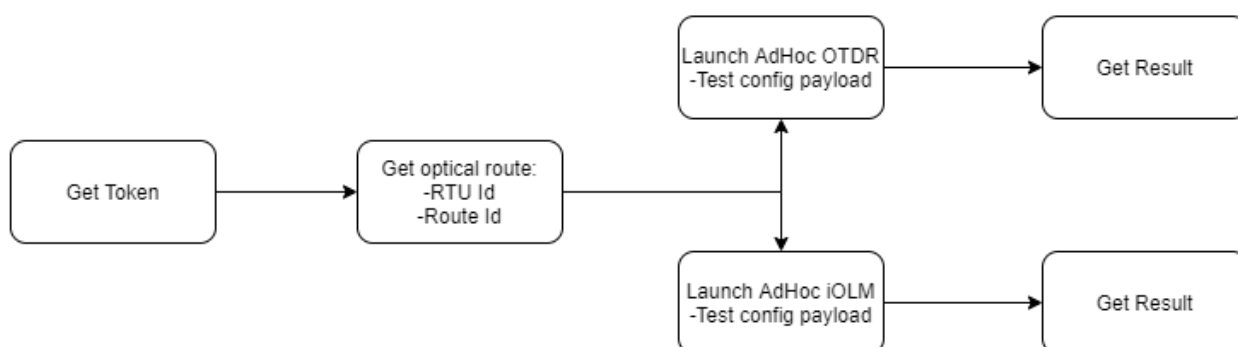
Postman collection

File names: UC_Topo_iOLM_Tests_PON.postman_collection.json

PON workflow API.postman_collection.json

USE CASE - How to run Ad Hoc iOLM or OTDR test

Workflow



Steps

1- Getting optical route

If the ID of targeted optical route is known, use this endpoint to have details on your route:

Endpoint: [#/opticalroutes/getOpticalRoute](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoute)

Else, search your Optical Route with a get all command.

Endpoint: [#/opticalroutes/getOpticalRoutes2](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoutes2) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoutes2).

Response:

Contains the information related on optical routes found.

Copy the optical route ID and the RTU ID to be able to launch the next request.

2 - Launching an AdHoc command

There are 2 endpoints: one for OTDR AdHoc test and another one for iOLM Adhoc test.

This type of test is conducted to check the health of fibers when required.

a/ Launching an OTDR AdHoc test

Endpoint: [#/control/commandOtdrAdHocTest](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandOtdrAdHocTest) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandOtdrAdHocTest).

b/ Launching an iOLM AdHoc test

Endpoint: [#/control/commandIolmAdHocTest](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandIolmAdHocTest) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandIolmAdHocTest).

Response:

Contains the promiseld to follow and retrieve the result.

Copy this promiseld to get result

3 - Launching get result

Endpoint: [#/Results/get_results](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Results/get_results) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Results/get_results).

You can perform searches in result entries using supported OData query options (see Result consulting use case for more option details).

Apply a filter based on opticalroute ID, test category and the promiseld to retrieve the searched result.

```
$filter= metadata/AssetId eq {{opticalRouteId}} and metadata/TestCategory eq 'Adhoc' and metadata/PromiseId eq {{PromiseId}}
```

Postman collection

File name: UC_Test_AdhocTests.postman_collection.json

Overview - When to update test setup of a route

There are three different situations where a test setup update is needed. Intent and workflow are very different.

1. Set the **link definition** to match physical configuration of the link, e.g.: changing splitter ratio.
2. In this case, any existing baseline must be deleted first ('renew' baseline via Delete command)
3. The new link definition will be detected by iOLM by the new baseline.
4. HRDs that were detected before the "renew" action will not be matched in the fresh baseline.
5. It is important to start the optical route life cycle with the right link definition.
6. Change the **thresholds** to be applied on link elements (during future update baselines), HRD test or monitoring thresholds applied in monitoring and test on demand.
7. Change the **Golden Trace** settings to increase OTDR resolution in a specific part of the optical route.
8. In this case, an update baseline operation is needed to refresh iOLM trace set, but all elements previously identified in the baseline are kept (F1 events, splitter, HRDs)

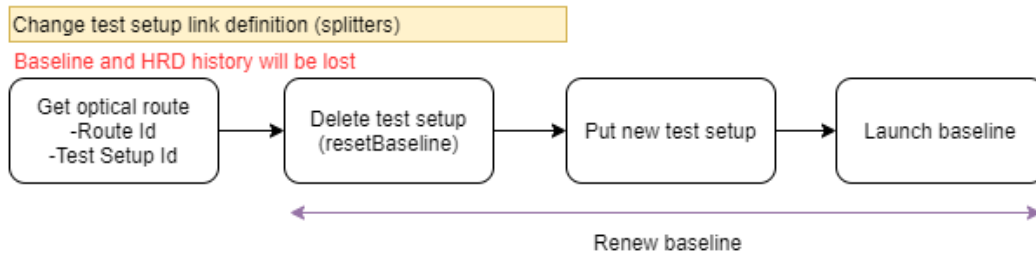
It is important to note that each update baseline will reset the previously detected link and element loss reference. Deviation returned in the next monitoring or TOD will be related to last update baseline. An update baseline is an acknowledgement of the actual state of the link and should not be launched while the optical route is in fault and waiting for repair.

The default test setup payload for a PON optical route can be obtained from the following endpoint.

[#/testconfigurations/default?monitoringType=pon&testCategory=monitoring](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/testconfigurations/default?monitoringType=pon&testCategory=monitoring) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/testconfigurations).

See below the 3 different cases.

USE CASE - Change link definition in test setup and update baseline



Steps

1- Getting optical route

If the ID of targeted optical route is known, use this endpoint to have details on your route:

Endpoint: [#/opticalroutes/getOpticalRoute](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoute).

Else, search your optical route with a get all command.

Endpoint: [#/opticalroutes/getOpticalRoutes2](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoutes2).

Response:

Contains the information related on optical routes found.

Copy the optical route ID and the RTU ID to be able to launch the next request.

2 - Deleting the current baseline

Endpoint: [#/control/resetBaseline](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/resetBaseline).

When this request is performed, the current baseline is deleted.

3 - Defining a new test setup

Endpoint: [#/opticalroutes/putTestSetup](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/putTestSetup).

To change the test setup link definition, you can refer to the document in appendix that explains payloads and measures.

4 - Launching a new baseline request

Endpoint: [#/control/commandBuild](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandBuildOnDemandTest).

Response:

Contains the promiseld to follow and retrieve the result.

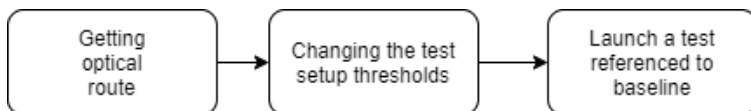
Copy this promiseld to get result

Postman collection

File name: UC_Topo_TestSetupLinkDefinition_TestSPON.postman_collection.json

USE CASE - Change thresholds in test setup

Workflow



Steps

1- Getting optical route

If the ID of targeted optical route is known, use this endpoint to have details on your route:

Endpoint: [#/opticalroutes/getOpticalRoute](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoute)

Else, search your optical route with a get all command.

Endpoint: [#/opticalroutes/getOpticalRoutes2](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoutes2)

Response:

Contains the information related on optical routes found.

Copy the optical route ID, the RTU ID and the test setup ID to be able to launch the next request.

2 - Changing the test setup thresholds

Endpoint: [#/opticalroutes/putTestSetup](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/putTestSetup)

To change the test setup link definition, you can refer to the document in appendix that explains payloads and measures.

3 - Optionally, launch a test that is referenced to baseline

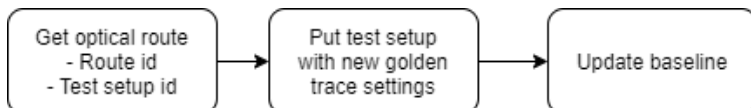
The next test that is referred to test setup will use updated thresholds: Baseline, TOD, Monitor, AdHorRD.

Postman collection

File name: UC_Topo_ChangeTestSetupThresholds_TestsPON.postman_collection.json

USE CASE - Change “Golden Trace” pulse and duration and update baseline

Workflow



Steps

1- Getting optical route

If the ID of targeted optical route is known, use this endpoint to have details on your route:

Endpoint: [#/opticalroutes/getOpticalRoute](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoute)

Else, search your optical route with a get all command.

Endpoint: [#/opticalroutes/getOpticalRoutes2](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoutes2) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoutes2)

Response:

Contains the information related on optical routes found.

Copy the optical route ID and the RTU ID to be able to launch the next request.

2 - Changing the test setup with new golden trace settings

Endpoint: [#/opticalroutes/putTestSetup](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/putTestSetup) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/putTestSetup)

To change the test setup link definition, you can refer to the document in appendix that explains payloads and measures.

3 - Launching an update baseline request

Endpoint: [#/control/commandBuild](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandBuildOnDemandTest) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandBuildOnDemandTest)

Response:

Contains the promiseld to follow and retrieve the result.

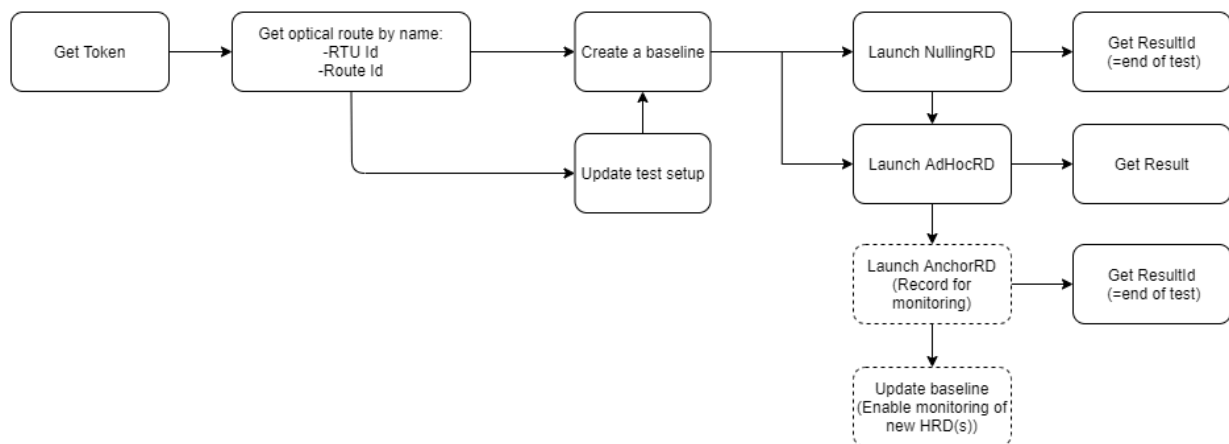
Copy this promiseld to get result

Postman collection

File name: UC_Topo_ChangeGoldenTraceSettings_TestsPON.postman_collection.json

USE CASE - How to launch a HRD test using OpticalRouteName

Workflow



Steps

1- Getting optical route

Search your optical route with a get all command using the optical route name as filter.

For more information, you can see the dedicated use case about that.

Endpoint: [#/opticalroutes/getOpticalRoutes2](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoutes2) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoutes2)

Fields:

- **name:** This string parameter allows to limit the number of returned results. The pattern of filtering used here is the contains mode. A full or a partial name can be used.
- **pageSize:** number max of optical route returned by page

- **pageNumber:** page number

Contains the information related on optical routes found.

Copy the optical route ID and the RTU ID to be able to launch the next request.

2 - Updating the default test setup if needed

Endpoint: [#/opticalroutes/putTestSetup \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/putTestSetup\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/putTestSetup)

To configure a test setup, you can refer to the document in appendix that explains payloads and measures.

3 - Creating a baseline if needed

Endpoint: [#/control/commandBuild \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandBuildOnDemandTest\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandBuildOnDemandTest)

The payload shall contain the baseline operationType__:

__ "MeasurementType": "PONXplorer;OperationType=Baseline"

Response:

Contains the promiseld to follow and retrieve the result.

Copy this promiseld to get result

4 - Launching a HRD related command

Use the following operations for HRD installation:

- RLNulling Refresh of peak reference (eliminates any possible ambiguity for single HRD detection)
- AdHocRD HRD detection and measurement
- AnchorRD Record the detected HRD for monitoring. Works only if last test is AdHocRD

These operations are launched with the same endpoint.

Endpoint: [#/control/commandBuild \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandBuildOnDemandTest\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandBuildOnDemandTest)

The specific operation is passed through payload

- "MeasurementType": "PONXplorer;OperationType=AdHocRD"
- "MeasurementType": "PONXplorer;OperationType=RLNulling"
- "MeasurementType": "PONXplorer;OperationType=AnchorRD"

The PortId is needed for AnchorRD, and must be the same as the AdHocRD result to be recorded.

It is possible to launch an AdHocRD test without PortId to do a simple continuity test when there is no intent of monitoring or reporting.

5 - Launching an update baseline request

Endpoint: [#/control/commandBuild \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandBuildOnDemandTest\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandBuildOnDemandTest)

Same request as for a baseline creation.

The payload shall contain the Baseline operationType__.

Response:

Contains the promiseld to follow and retrieve the result.

Copy this promiseld to get result

Postman collection

File name: UC_TestPON_HRD_Discover.postman_collection.json

USE CASE - Moving an HRD to another terminal port

Two scenarios are supported:

1. The HRD is moved to a port that is at a similar optical distance (40cm) and a different PortId is used to launch the AdHocRD test.
2. The HRD is moved by a greater optical distance but the same PortId is used to launch AdHocRD

In both scenarios, anchoring HRD will replace previous HRD record.

Workflow

Disconnect HRD from port A

Refresh peak reference (RLNulling)

Launch AdHocRD

-PortId: A or

-PortId: B

Connect HRD to new port B

Send AnchorRD

-PortId

Update Baseline

Steps

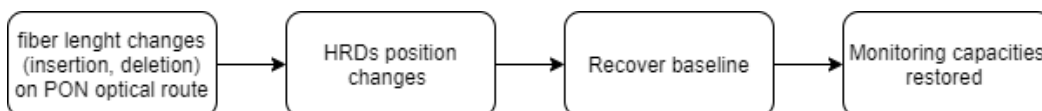
The steps are the same as HRD discover and anchor. In this process, the Refresh Peak Reference step is mandatory after removing the HRD, and before connecting it to the new port.

Postman collection

File name: **UC_TestPON_MovingHRD.postman_collection.json**

USE CASE - Basic recovery baselines and HRDs after link repair

Workflow



After equipment replacement or link reparation (insertion, deletion of fiber section), this option allows you to restore monitoring capacities on a PON route without reinstalling each HRD.

Steps

1- Getting optical route

If the ID of targeted optical route is known, use this endpoint to have details on your route:

Endpoint: [#/opticalroutes/getOpticalRoute \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoute\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoute)

Else, search your optical route with a get all command.

Endpoint: [#/opticalroutes/getOpticalRoutes2 \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoutes2\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoutes2)

Response:

Contains the information related on optical routes found.

Copy the optical route ID and the RTU ID to be able to launch the next request.

2 - Launching a recover baseline request

Endpoint: [#/control/commandBuild \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandBuildOnDemandTest\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandBuildOnDemandTest)

The payload shall contain the baseline operationType__:__

__ "MeasurementType": "PONXplorer;OperationType=RecoverBaseline"

Response:

Contains the promiseId to follow and retrieve the result.

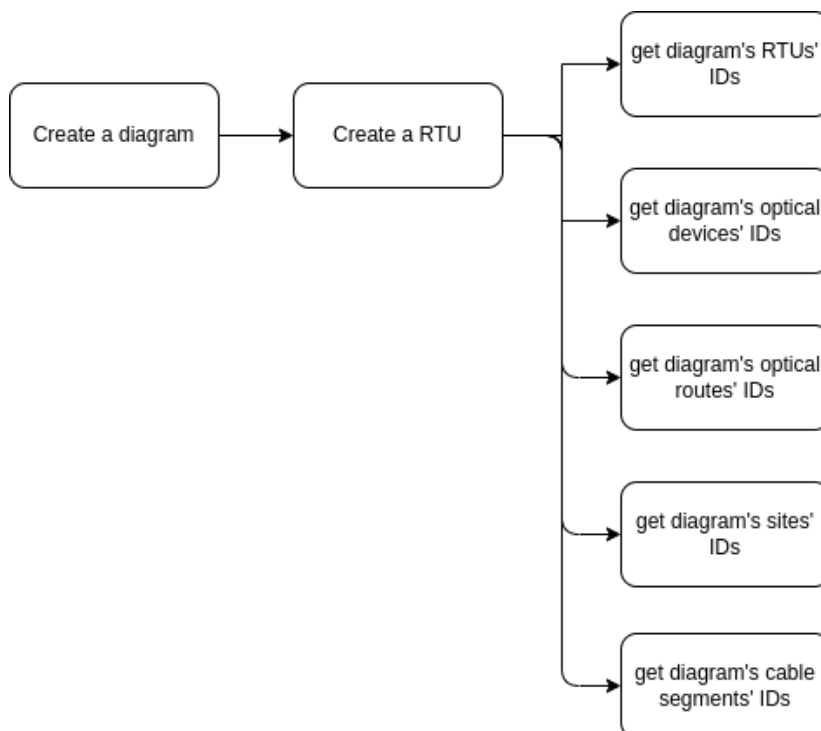
Copy this promiseId to get result.

Postman collection

File name: UC_Topo_RecoverBaseline.postman_collection.json

USE CASE - Diagram populating and content retrieving

Workflow



Steps

Diagrams are a grouping mechanism of topological objects. Since FMS version 8.0, RTU(s) must be grouped under diagram(s), at least a default one.

Create a diagram

Endpoint: [#/diagrams/createDiagram \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/diagrams/createDiagram\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/diagrams/createDiagram)

Response:

This call will return the created diagram with its unique ID.

Use the provided ID to create RTUs.

Create a RTU owned by the diagram

Set the owning diagram ID in the payload and perform the following call.

Endpoint: [#/remotetestunits/createRtu](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/remotetestunits/createRtu) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/remotetestunits/createRtu).

Get diagram's RTUs' IDs

Set the diagram ID and perform the following call.

Endpoint: [#/diagrams/getRtuIds](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/diagrams/getRtuIds) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/diagrams/getRtuIds).

Get diagram's optical devices' IDs

Set the diagram ID and perform the following call.

Endpoint: [#/diagrams/getOpticalDeviceIds](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/diagrams/getOpticalDeviceIds) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/diagrams/getOpticalDeviceIds).

Get diagram's optical routes' IDs

Set the diagram ID and perform the following call.

Endpoint: [#/diagrams/getOpticalRouteIds](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/diagrams/getOpticalRouteIds) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/diagrams/getOpticalRouteIds).

Get diagram's sites' IDs

Set the diagram ID and perform the following call.

Endpoint: [#/diagrams/getSiteIds](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/diagrams/getSiteIds) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/diagrams/getSiteIds).

Get diagram's cable segments' IDs

Set the diagram ID and perform the following call.

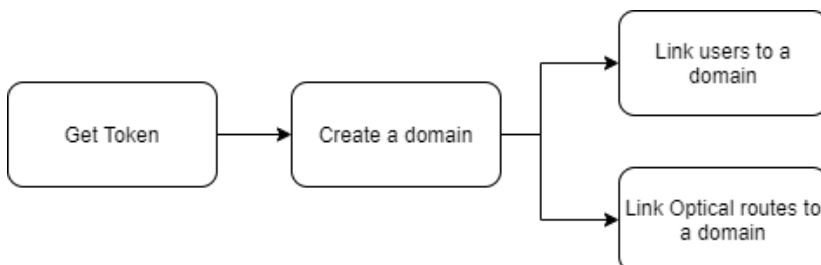
Endpoint: [#/diagrams/getCableSegmentIds](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/diagrams/getCableSegmentIds) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/diagrams/getCableSegmentIds).

Postman collection

File name: UC_Topo_Diagram_Configuration.postman_collection.json

USE CASE - Domain configuration for alarm notification

Workflow



Steps

Domains are a logical grouping mechanism supporting alarm notification in relation to users and monitored optical routes.

1 - Creating a Domain

Endpoint: [#/domains/createDomain \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/domains/createDomain\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/domains/createDomain)

Response:

This call will return the created domain with its unique ID.

Use the provided ID to create user accesses and link optical routes.

2 - Link Optical Routes to a Domain

Set the optical routes ID and perform the following call.

Endpoint: [#/domains/addContentsToDomain \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/domains/addContentsToDomain\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/domains/addContentsToDomain)

How to retrieve the optical route ID to create my domain?

1 - You can select the routes related to an RTU:

Endpoint: [#/remotetestunits/getRtus \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/remotetestunits/getRtus\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/remotetestunits/getRtus)

The IDs of associated optical routes are joined in response (see monitored assets content part).

2 - You can filter by diagram if diagrams are defined in FMS.

Endpoint: [#/diagrams/getAllDiagrams \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/diagrams/getAllDiagrams\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/diagrams/getAllDiagrams)

Retrieves all diagrams up to max_get_size elements.

Without pagination, it returns an array of diagrams:

```
[
  { "diagram": { ... }
}
```

For get request without parameter ('getAll'), two headers are added in the response:

- 'TotalObjectsCount': indicates the total number of diagrams in the database.
- 'PartialResult': indicates whether or not all elements have been returned.

Since getAll request is limited by max_get_size, the field 'PartialResult' will be set to true if the response has been capped.

Using pagination, it returns:

- a json array of 'objects' containing the diagrams.
- totalObjectCount: indicates the total number of diagrams present in the database.
- totalPageCount: indicates the total number of pages presuming each page has max_get_size elements.

```
{
  "objects": [
    { "diagram": { ... }
  },
  "totalPageCount": int,
  "totalObjectCount": int
}
```

If pageSize is higher than max_page_size, it returns HTTP 400. max_get_size and max_page_size can be configured/overridden with config file /opt/fms/solution/config/topology/endpoints_size.properties.

3 - Get all optical routes and make your choice.

Endpoint: [#/opticalroutes/getOpticalRoutes2 \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoutes2\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/opticalroutes/getOpticalRoutes2)

3 - Link users to a domain

With each user account defined in IAM, perform the following call on the API to enable email notification reception.

Endpoint : [#/domains/addUserAccesses \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/domains/addUserAccesses\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/domains/addUserAccesses)

Ensure that the user account has an email address configured.

Values for **accessLevels** parameter:

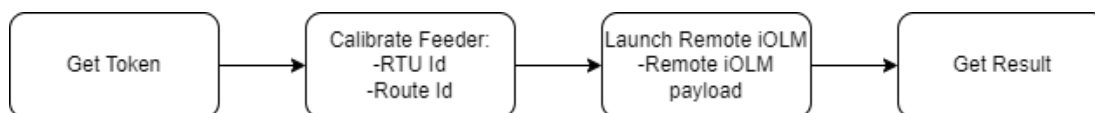
- * NOTIFICATION = any alarm severity
- * NOTIFICATION-CRITICAL = critical alarm severity
- * NOTIFICATION-MAJOR = major alarm severity
- * NOTIFICATION-MINOR = minor alarm severity
- * NOTIFICATION-WARNING = warning alarm severity
- * NOTIFICATION-CLEARED = cleared alarm severity
- * NOTIFICATION-INDETERMINATE = indeterminate alarm severity

Postman collection

File name: UC_Topo_Domain_Configuration.postman_collection.json

USE CASE - To calculate and send LinkLength and LinkLoss value for Remote iOLM test based on zero position harmonization

Workflow



Steps

1 - Calibrate feeder using feeder calibration API

2 - Launch Remote iOLM test based on feeder calibration data

Endpoint: [#/control/commandIolmAdHocTest](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/commandIolmAdHocTest)

With introduction of zero position on the remote switch, the LaunchFiberLength and Loss sent as part of Remote iOLM test payload should be sent based on below calculations taken from calibrated feeder:

1 - LaunchFiberLength = brief.LinkResults.Length - brief.Measurement.Elements[0].Position

2 - Loss = brief.Measurement.Elements[0].Results[0].CurveLevel - brief.Measurement.Elements[endelement].Results[0].CurveLevel

Note: The above details of LaunchFiberLength and Loss are supported on versions FMS 8.0 and RTU 2.13. Also, these calculations for Remote iOLM test are taken care internally in smart app application.

Response:

Contains the promiseId to follow and retrieve the result.

Copy this promiseId to get result

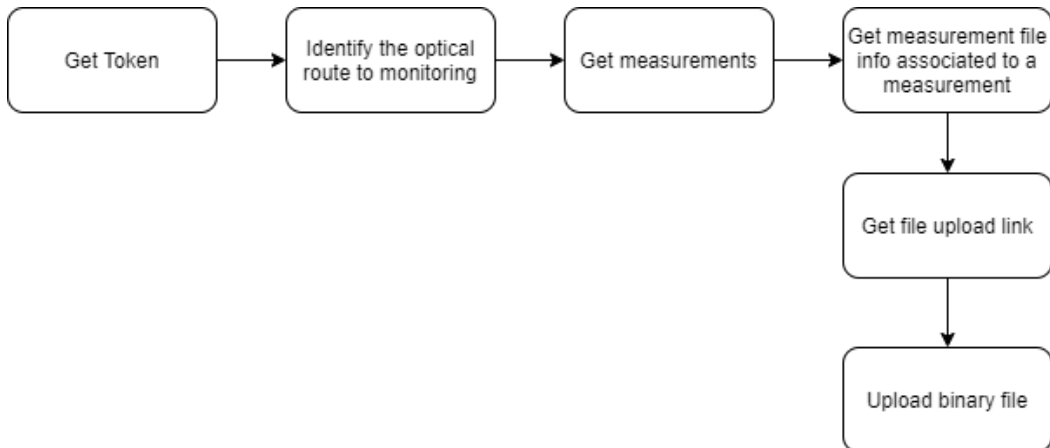
Measurement data & measurement files APIs

Roles

fg-results-master	True	all permissions for T&M Services
fg-results-read	True	read access for T&M Services

USE CASE - How to view the measures - Result Consulting

Workflow



Result Filtering

Get measurements

Endpoint: [#/Results/get_results \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Results/get_results\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Results/get_results)

You can perform searches in result entries using supported OData query options.

Possibility to:

- Filter out your results by using \$filter query option
- Project your results by using \$select query option
- Sort your results using \$orderby query option
- Perform paging on results using \$top & \$skip query options

Filtering by assetId (template)

\$filter=metadata/AssetId eq < opticalRouteId >

Filtering limited to a testCategory (template)

\$filter=metadata/AssetId eq < opticalRouteId > and metadata/TestCategory eq 'VeryFirstReference'

Limiting the information returned with \$select (template)

\$filter=metadata/AssetId eq < opticalRouteId > &\$select=resultid,metadata,status

Limiting the number of measurements returned with \$top (template)

\$filter=metadata/AssetId eq < opticalRouteId > &\$top=10

Getting the last measurement with \$orderby to sort by testTime (template)

\$filter=metadata/AssetId eq < opticalRouteId > &\$top=1_&_ \$orderby= metadata/TestTime desc

The metadata of measurement contains all information on the monitoring context: HasFault, AssetId, BaselineId, PromiseId and so on.

Getting measurement file information

To retrieve the file information associated to a measure, use the resultid contained in the measurement and call the following endpoint:

Endpoint: [#/Files/get_results_resultId_files \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Files/get_results_resultId_files\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Files/get_results_resultId_files)

Response:

Contains the file name, the file ID, the status...

The file ID is important for the next step.

Getting the file link to upload the measurement binary file

Retrieve the file link of measurement binary file.

Endpoint: [#/Files/get_results_resultId_files_fileId_link](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Files/get_results_resultId_files_fileId_link)

Response:

Contains the link to upload the measurement file.

Uploading the binary file

Retrieve the file link of measurement binary file.

Endpoint: [#/Client/get_clientDataDir_filePath_](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Client/get_clientDataDir_filePath_)

Replace the local URI <http://measurement-files:9080> (<http://measurement-files:9080>) with the measurement file address `https://<data.fms.local>/upload`

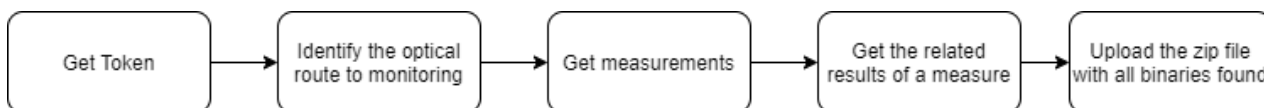
Response:

Contains the binary file.

Postman collection

File name: UC_Data_Result_Queries.postman_collection.json

USE CASE - How to upload a zip with the measures related to your current measure

Workflow**Steps****1 - Get measurements**

Endpoint: [#/Results/get_results](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Results/get_results)

You can perform searches in result entries using supported OData query options.

Possibility to:

- Filter out your results by using \$filter query option
- Project your results by using \$select query option
- Sort your results using \$orderby query option
- Perform paging on results using \$top & \$skip query options

Filtering by assetId (template)

`$filter=metadata/AssetId eq < opticalRouteId >`

Filtering limited to a testCategory (template)

`$filter=metadata/AssetId eq < opticalRouteId > and metadata/TestCategory eq 'VeryFirstReference'`

Limiting the information returned with \$select (template)

\$filter=metadata/AssetId eq < opticalRouteId > &\$select=resultId,metadata,status

Limiting the number of measurements returned with \$top (template)

\$filter=metadata/AssetId eq < opticalRouteId > &\$top=10

Getting the last measurement with \$orderby to sort by testTime (template)

\$filter=metadata/AssetId eq < opticalRouteId > &\$top=1__&__orderby= metadata/TestTime desc

The metadata of measurement contains all information on the monitoring context: HasFault, AssetId, BaselineId, PromiseId and so on.

2 - Get the related results of a measure

Endpoint: [#/Results/get_results_resultId_relatedresults \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Results/get_results_resultId_relatedresults\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Results/get_results_resultId_relatedresults)

Get a list of resultIds associated to the current measurement:

- For OTDR and IOLM get the most recent VeryFirstReference/Baseline,
- For IOLM PON get the Baseline, the RLNulling as well as all related measures that are present on the FMS server.

Response:

Contains an array of result IDs. The first one is the ID of the current measurement.

3 - Upload the zip file with all binaries found

Endpoint: [#/Client/get_clientDataDir_zip \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Client/get_clientDataDir_zip\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Client/get_clientDataDir_zip)

Takes a list of result IDs (maximum 20 IDs) and returns their corresponding files zipped into one file. The zip file carries the same name as the file associated with the first result ID of the list passed in parameters.

Response:

Contains the zip file.

Postman collection

File name: UC_Data_Measurement_Zip.postman_collection.json

USE CASE - How to extract OTDR traces from an iOLM measure and retrieve them in a zip folder**Workflow****Steps****1 - Get iOLM measurements**

Endpoint: [#/Results/get_results \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Results/get_results\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Results/get_results)

perform a searches in result entries to retrieve an iOLM measure

Filtering by TestType (template)

\$filter=metadata/TestType eq "iOLM"

Filtering by assetId (template)

\$filter=metadata/AssetId eq < opticalRouteId >

Before passing on the next step set the global variable 'resultId' to the result you found.

2 - Extract OTDR from the iolm measurement id

Endpoint: [#/Results/post_results_resultId_otdr_extract](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Results/post_results_resultId_otdr_extract) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Results/post_results_resultId_otdr_extract)

Extract all OTDR traces from an iOLM measure

Response:

Contains a success message and an array of the OTDR results IDs.

3- Get the related results of a measure

Endpoint: [#/Results/get_results_resultId_relatedresults](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Results/get_results_resultId_relatedresults) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Results/get_results_resultId_relatedresults)

Get a list of resultsIds associated to the current measurement:

Response:

Contains an array of result IDs. The first one is the ID of the current measurement.

4- Upload the zip file with all binaries found

Endpoint: [#/Client/get_clientDataDir_zip](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Client/get_clientDataDir_zip) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/Client/get_clientDataDir_zip)

Takes a list of result IDs (maximum 20 IDs) and returns their corresponding files zipped into one file. The zip file carries the same name as the file associated with the first result ID of the list passed in parameters.

Response:

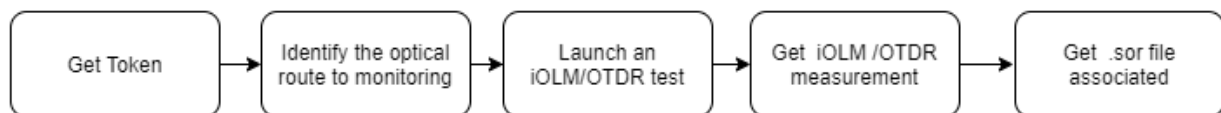
Contains the zip file.

Postman Collection

File name: UC_Data_OTDR_Extraction.postman_collection.json

USE CASE - How to retrieve the .sor file in binary or Base64 from a resultId of an OTDR result

Workflow



Steps

1 - Launch an iOLM/OTDR test command

After launching a test request on the targeted optical route, you will receive a promiseId in the synchronous response body.

The test has not yet been carried out but the RTU has notified that the request has been taken into account. A notification of the test result will be sent asynchronously on the websocket (and on measurement-kafka-router or notification-handler addons in some types of FMS deployment).

For more details, please refer to the document 'RTU errors flow.pdf'

Response:

Contains the PromiseId to follow and retrieve the result.

Copy this PromiseId to get result

2 - Get iOLM/OTDR measurements result

Endpoint: [\[#/Results/get_results\]](https://d92wagemn7fj.cloudfront.net/topology-api/master/latest/index.html) (<https://d92wagemn7fj.cloudfront.net/topology-api/master/latest/index.html>) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html)" \ | "/Results/get_results)

Perform a searches in result entries to retrieve the measure

Filtering by TestType, AssetId and promiseId (template)

\$filter= metadata/AssetId eq and metadata/TestType eq "" and metadata/PromiseId eq "

For iOLM result, copy the resultid to the result and the resultid of relatedresult (OTDR trace) in result metadata.

Endpoint: [#/Results/get_results_otdr_sorfile](https://d92waqemn7fj.cloudfront.net/topology-api/master/latest/index.html) (https://d92waqemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html)" \ "(/Results/get_results_otdr_sorfile)

- For OTDR result,
 - set the 'OtdrResultId' parameter with the resultId saved to the previous step.
- For iOLM result (e.g baseline, AdHocRD...),
 - set the 'OlmResultId' parameter with the resultId of measurement
 - set the 'OtdrResultId' parameter with the resultId of related result (OTDR trace).

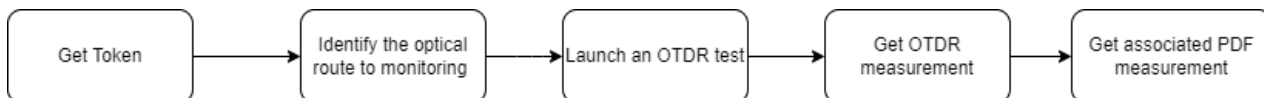
Base64 format example:

Binary format example:

For unhappy paths, the HTTPS errors codes are listed in swagger documentation.

File name: UC Data Get SOR File.postman collection.json

Workflow



1 - Launch an iOLM/OTDR test command

After launching a test request on the targeted optical route, you will receive a `Promise` in the synchronous response body.

The test has not yet been carried out but the RTU has notified that the request has been taken into account. A notification of the test result will be sent asynchronously on the websocket (and on measurement-kafka-router or notification-handler addons in some types of FMS deployment).

For more details, please refer to the document 'RTU errors flow.pdf'

Response:

Contains the PromiseId to follow and retrieve the result.

Copy this Promiseld to get result

2 - Get OTDR measurements result

Endpoint: [/Results/get_results](https://d92wagemn7fj.cloudfront.net/topology-api/master/latest/index.html) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html)" | "/Results/get_results)

Perform a searches in result entries to retrieve the measure

Filtering by TestType, AssetId and promiseId (template)

```
$filter= metadata/AssetId eq and metadata/TestType eq "" and metadata/PromiseId eq "
```

TestType : OTDR

For OTDR result, copy the 'resultid' to the result.

3 - Launch a pdf command

After launching a command, response will be PDF report from a measurement. Currently support for OTDR measurement.

Endpoint: [#/Results/get_PDF_report](https://d92wagemn7fj.cloudfront.net/topology-api/master/latest/index.html) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html)" \ | "/Results/get_PDF_report)

Response:

In response, you will get the pdf file.

The screenshot shows a Postman interface with a GET request to the endpoint: `{{Data_Host}}/v1/results/pdf?resultId=7c1209f6-2db1-41cc-af09-2a8ca5b9fcc2`. The response is a PDF report titled "OTDR Report" with a status of "Fail". The report content is as follows:

General Information	
Filename:	FG7501002849_Learning_FG88-H1_2023-10-10_09-30-27_460.sor
Test date:	10/10/2023
Test time:	09:30:28 (UTC)
Cable ID:	
Fiber ID:	
Location A:	
Comments:	
Job ID:	
Customer:	
Company:	
Location B:	
None:	

Location A	Location B
Location:	Location:
Operator:	Operator:
Unit model:	Unit model:
Unit s/n:	Unit s/n:
Calibration Date:	Calibration Date:

Test Parameters	
Wavelength (nm):	1625 nm
Range (km):	20.9997
Pulse (ns):	100
Duration (s):	15

For unhappy paths, the HTTPS errors codes are listed in swagger documentation.

Postman Collection

File name: UC_Get_PDF_report.postman_collection.json

USE CASE - To Add / update birth certificate of an OTDR result

Roles

Role Name	Composite	Description
measurement-BC-create	False	permission to create birth certificate for a measurement
measurement-BC-update	False	permission to update birth certificate for a measurement

Workflow



Steps

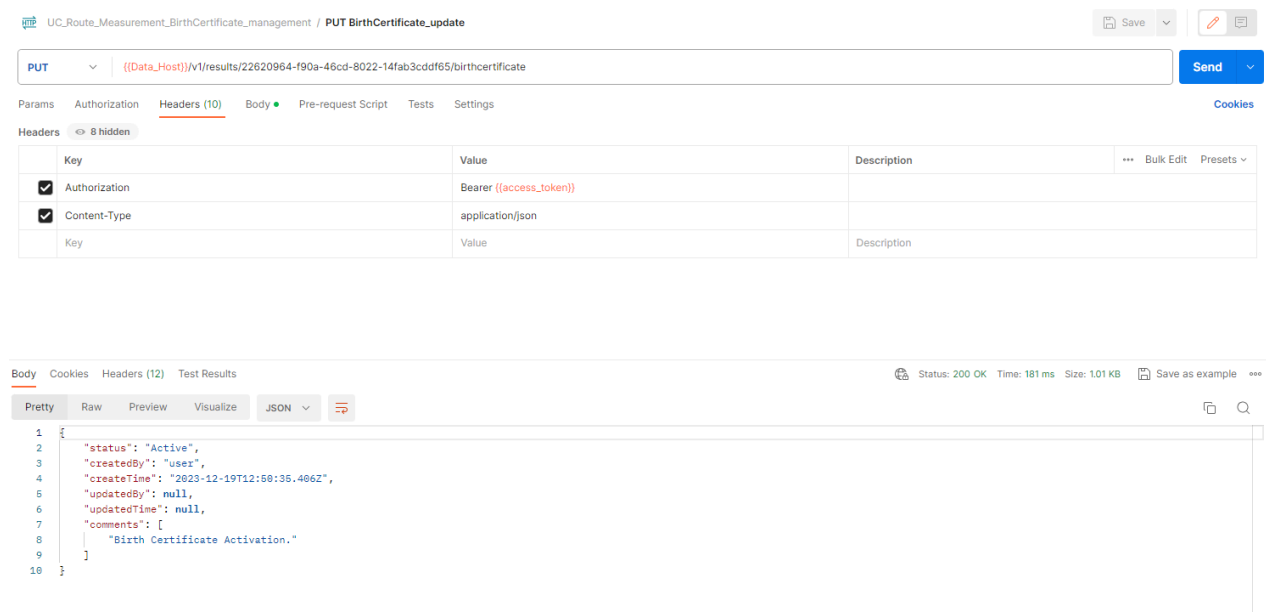
- 1 - Identify optical route for which birth certificate creation is required
- 2 - Identify measurement which will be birth certificate of optical route
- 3 - Launch a birth certificate PUT call

After launching a command, response will be HTTP 204 report from a measurement. Currently support for OTDR measurement.

Endpoint: [#/Results/tag_birth_certificate_PUT]("https://d92wagemn7fj.cloudfront.net/topology-api/master/b7fc77f047f7753b7b49f7e8ac9c211d193ce1a9/index.html (https://d92wagemn7fj.cloudfront.net/topology-api/master/b7fc77f047f7753b7b49f7e8ac9c211d193ce1a9/index.html)" \ | "/Results/tag_birth_certificate_PUT)

Response:

In response, you will get the birth certificate JSON.



UC_Route_Measurement_BirthCertificate_management / PUT BirthCertificate_update

PUT {{Data_Host}}/v1/results/22620964-f90a-46cd-8022-14fab3cddf65/birthcertificate

Params Authorization Headers (10) Body Pre-request Script Tests Settings Cookies

Key	Value	Description
Authorization	Bearer {{access_token}}	
Content-Type	application/json	
Key	Value	Description

Body Cookies Headers (12) Test Results

Status: 200 OK Time: 181 ms Size: 1.01 KB Save as example

```

1 {
2   "status": "Active",
3   "createdBy": "user",
4   "createTime": "2023-12-19T12:50:35.406Z",
5   "updatedBy": null,
6   "updateTime": null,
7   "comments": [
8     "Birth Certificate Activation."
9   ]
10 }

```

For unhappy paths, the HTTPS errors codes are listed in swagger documentation.

Postman Collection

File name: UC_Route_Measurement_BirthCertificate_management.json

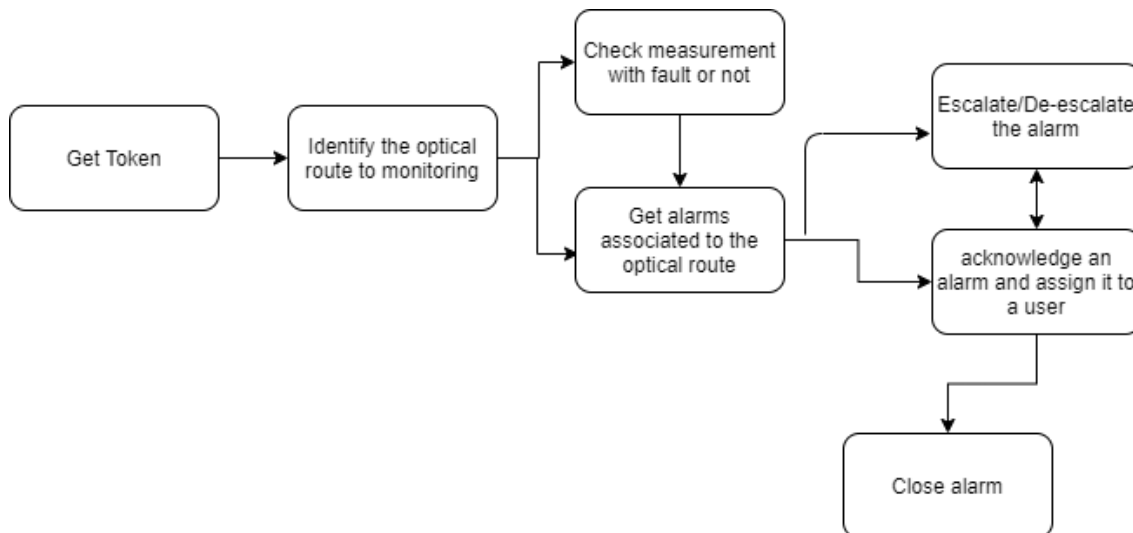
Alarm API

Roles

Role Name	Composite	Description
fg-alarm-acting	True	permission to update an alarm with respect to alarm life cycle.
fg-alarm-all	True	permissions update any active alarm.
fg-alarm-master	True	all permissions for alarms that belong to tracking services.

Alarms consulting

Workflow



Alarms filtering

Get alarms

Endpoint: [#/alarms/getAlarms](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/alarms/getAlarms).

You can perform searches in result entries using supported OData query options.

Possibility to:

- Filter out your alarms by using \$filter query option
- Project your alarms by using \$select query option
- Sort your alarms using \$orderby query option
- Perform paging on alarms using \$top & \$skip query options

Filtering by assetId (template)

\$filter=sources/assetId eq

Limit the number of measurements returned with \$top (template)

\$filter=metadata/AssetId eq & \$top=10

Get the new major alarms (template)

\$filter=metadata/AssetId eq and alarmState/state eq 'new' and severity eq 'major'

Escalate / deescalate alarms / acknowledge an alarm and assign it to a user / close alarms

Endpoint: [#/alarms/putAlarm](#) (https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/alarms/putAlarm).

It's the same endpoint for all changes to the alarms:

- Change the severity
- Assign a user

Event of type stateChanged, severityChanged, assignmentChanged will be generated automatically.

Prerequisite to close an alarm: The alarm shall be acknowledged before.

You can retrieve some examples in the following postman collection.

Postman collection

File name: UC_Alarms.postman_collection.json

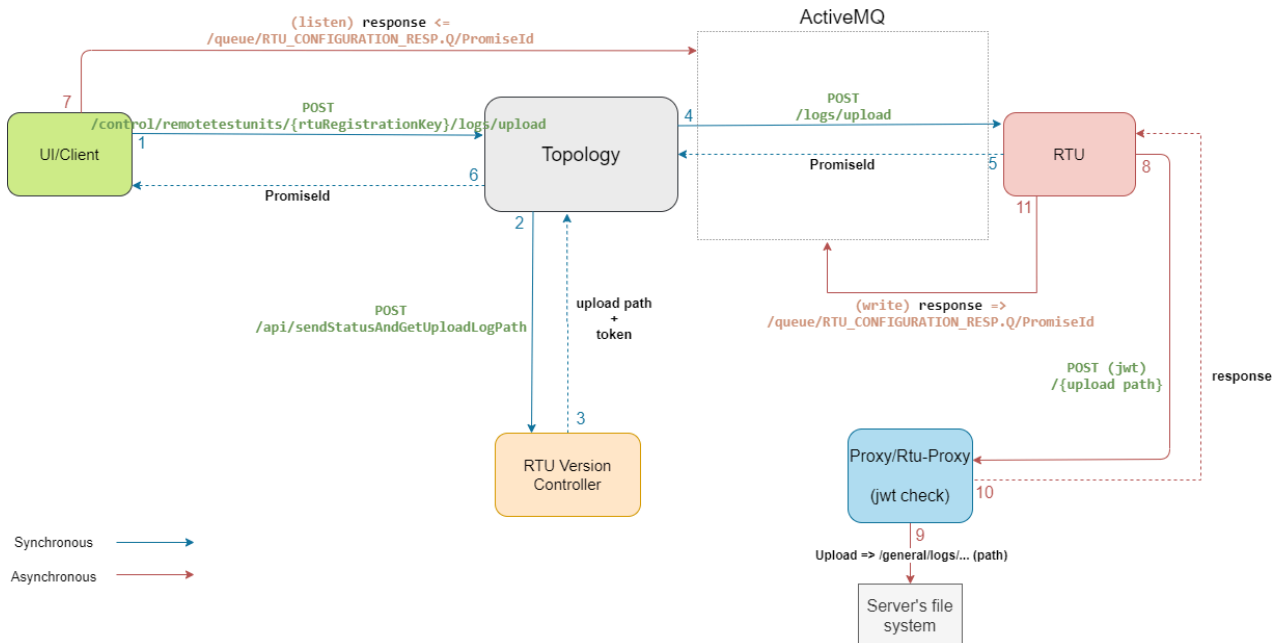
RTU logs consulting

It is possible to request a given RTU logs as well set its log level. The RTU must be at least registered.

Upload logs from RTU to file system

Endpoint: [#/logs/upload \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/requestRtuLogsUpdate\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/requestRtuLogsUpdate).

Workflow



Needs

- rtuRegistrationKey: the registration key from the registered RTU

Change the log level verbosity

Endpoint: [#/logs \(https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/requestRtuLogsLevel\)](https://d92wagemn7fj.cloudfront.net/topology-api/Release_8.5.0/4f57760d6ecd0461d71a7dcde575e186dc4b125e/index.html#/control/requestRtuLogsLevel).

Query parameters:

- level: (required) the new log level verbosity. Value can be: Critical, Error, Warning, Information, Debug, Trace.
- timeToKeepLogLevel: (optional) the time before the log level come back to "Warning" in seconds. Default = 1day.

Postman collection

File name: UC_Rtu_Logs.postman_collection.json

Notification handler addon API

Notification handler addon service, when activated, stores test result notifications for a certain amount of time (default is 2 days) and offers REST endpoints to get corresponding resultIds to retrieve the result files themselves (cf. **"USE CASE - How to retrieve the .sor file in binary or Base64 from a resultId of an OTDR result"** for more details on how to get result files from resultIds). This service also exposes notifications of error cases (unhappy paths).

Get all notifications

Endpoint: `{{proxyAddress}}/_notif/search`

Example of response when 2 notifications are stored:

```
{
  "took": 5,
  "timed_out": __false__,
  "_shards": {
    "total": 2,
    "successful": 2,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": 2,
    "max_score": 1.0,
    "hits": [
      {
        "_index": "notif-2021-09-16.12-25-01-000083",
        "_type": "_doc",
        "_id": "g7mT7nsBcJ2tBY7vFIQJ",
        "_score": 1.0,
        "_source": {
          "promiseId": "bdbd8944-ec21-4153-a81f-2f5c9ded8080",
          "resultId": "fb56a0fb-8f8b-4f9f-acc-30295fcc1f43"
        }
      },
      {
        "_index": "notif-2021-09-16.12-25-01-000083",
        "_type": "_doc",
        "_id": "hLmU7nsBcJ2tBY7vXITA",
        "_score": 1.0,
        "_source": {
          "promiseId": "0678f33e-0354-4889-b7c0-617b3ff90dab",
          "resultId": "74b64cda-47e8-41c5-b5ff-49dab7be3b7a"
        }
      }
    ]
  }
}
```

Notice that:

- number of notifications is given by **hits.total** element
- notification information (promiseId, resultId) is given by **_source** of each element in **hits.hits[]**

Example of response when no notification is stored:

```
{
  "took": 1,
  "timed_out": false,
  "_shards": {
    "total": 2,
    "successful": 2,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": 0,
    "max_score": null,
    "hits": []
  }
}
```

Get notification related to a promiseId

Endpoint: {{proxyAddress}}/_notif/search?q=

Parameters:

- Key: q, Value: promiseId

Example of response when corresponding notification is still stored:

```
{
  "took": 9,
  "timed_out": false,
  "_shards": {
    "total": 2,
    "successful": 2,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": 1,
    "max_score": 1.4384104,
    "hits": [
      {
        "_index": "notif-2021-09-16.16-57-10-000001",
        "_type": "_doc",
        "_id": "GMWQ73sBRJ_oTiiUo3UT",
        "_score": 1.4384104,
        "_source": {
          "promiseId": "8670f3d3-fcfb-4faf-949b-8d47bb060024",
          "resultId": "fd94df4f-b9e1-4951-b137-fe9746425e92"
        }
      }
    ]
  }
}
```

Example of response when the test request returns an error

```
{
  "took": 48,
  "timed_out": __false__,
  "_shards": {
    "total": 2,
    "successful": 2,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": 1,
    "max_score": 6.019864,
    "hits": [
      {
        "_index": "notif-2021-09-16.17-13-09-000002",
        "_type": "_doc",
        "_id": "HMW73sBRJ_oTiiUhXXr",
        "_score": 6.019864,
        "_source": {
          "promiseId": "db9833f0-660a-4386-93bb-7e5d4a498f4f",
          "isError": "true",
          "error": "{\code\":500,\exceptionInformation\":{\Data\":{\},\ExceptionTypeName\":\Metrino.Otdr.SignalPro
cessing.UnableToMeasureException, Metrino.Otdr.SignalProcessing, Version=1.4.0.0, Culture=neutral, PublicKeyToken=02be2af3f10d9
688\", \InnerException\":null, \Message\":\Could not find switch on switch detection trace\", \Source\":\Metrino.Olm.SignalPr
ocessing\", \StackTrace\":\ at Metrino.Olm.SignalProcessing.Monitoring.OlmNodeAnalysis.AnalyseSingleStageSwitchTraces(Single
PulseTraceCollection traces, Int32 switchStage)\n at Metrino.Olm.Instrument.Step.NodeSwitchAcquisitionStep.Execute()\n a
t Metrino.Olm.Instrument.AcquisitionContext.AcquisitionThread()\", \message\":\Could not find switch on switch detection trac
e\", \messageKey\":\111_UnableToMeasure\", \parameters\":null}"
        }
      }
    ]
  }
}
```

Notice that `_source` element contains **isError** and **error** elements instead of a `resultId`.

The complete description of notification formats are described in the Kafka router add-on documentation.

Postman collection

File name: UC_NOTIF_HANDLER.postman_collection.json

Service Authorization Proxy

Specifications: [Service-Authorization-Proxy \(https://d92wagemn7fj.cloudfront.net/topology-api/master/7b4042cd4da05f4625876629f28d1369eec71228/service-authorization-proxy.json\)](https://d92wagemn7fj.cloudfront.net/topology-api/master/7b4042cd4da05f4625876629f28d1369eec71228/service-authorization-proxy.json)

IAM

It concerns operation on Identity Access Manager (IAM) through service authorization proxy

Get users

Retrieve users on Keycloak. It will forward the request to Keycloak Admin API users.

For more information, see: [Official Keycloak Admin API users doc \(https://www.keycloak.org/docs-api/15.0/rest-api/index.html#_users_resource\)](https://www.keycloak.org/docs-api/15.0/rest-api/index.html#_users_resource)

Endpoint: [#/users \(https://d92waqemn7fj.cloudfront.net/topology-api/master/7b4042cd4da05f4625876629f28d1369eec71228/index.html?urls.primaryName=service-authorization-proxy#/Users/get_users\)](https://d92waqemn7fj.cloudfront.net/topology-api/master/7b4042cd4da05f4625876629f28d1369eec71228/index.html?urls.primaryName=service-authorization-proxy#/Users/get_users)

Query parameters:

- first: (optional) Defines the pagination offset where to start searching users
- max: (optional) Defines the maximum results size. Default = 100
- search: (optional) String that matches the beginning of the username, first or last name, or email
- username: (optional) String that matches the beginning of the username, or the complete username, if the "exact" parameter is true
- exact: (optional) Boolean which defines whether the "username" parameter must match exactly

Postman collection

File name: UC_Proxy_IAM_Users.postman_collection.json

Topology V2 API

Inventory Management

Workflow

Authentication > Request to external server > Reception of data > Backend processing

Get Rtus

Endpoint: `#/api/v2/topology/inventory/external/rtus`

Run a synchronisation between the actual FMS installation and an external server.

Response: The response is a list of Rtu from the specified external inventory.

Post Rtu

Endpoint: `#/api/v2/topology/inventory/external/rtus`

Response: Import an RTU from external inventory to FMS.

Postman Collection

File name: FMS_external_IMS_APIs.postman_collection.json